

Fakultät Informatik

Entwicklung und Konzeption eines Device-Integration-Plugins für zenLAB

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Lukas Spindler

Matrikelnummer 3728783

Erstgutachter:	Prof. Dr. Matthias Teßmann
Zweitgutachter:	Prof. Dr. Matthias Meitner
Betreuer:	Jonas Krammer
Unternehmen:	infoteam Software AG

© 2026

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Kurzdarstellung

Kurze Zusammenfassung der Arbeit, höchstens halbe Seite. Deutsche Fassung auch nötig, wenn die Arbeit auf Englisch angefertigt wird.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Inhaltsverzeichnis

Vorwort	1
1. Einleitung	2
1.1. Problemstellung	3
1.2. Forschungsfragen	4
1.3. Zielsetzung und Abgrenzung	5
1.4. Aufbau der Arbeit	6
2. Grundlagen	7
2.1. zenLAB und bestehende Systemlandschaft	7
2.1.1. Middleware- und Plugin-Architektur	7
2.1.2. DeviceAgents und AgentManager	8
2.1.3. TreeNode-Modell	9
2.1.4. Bestehende Client- und WebAPI-Grenze	10
2.2. Dynamische Geräteintegration	10
2.2.1. Integrationen, Registrierungen und Geräte	10
2.2.2. Registrierung und zentrale Geräteverwaltung	11
2.2.3. Lebenszyklus, Lease und Reconnect	12
2.3. Kommunikation verteilter Komponenten	13
2.3.1. Auftrags- und Ergebnisprinzip	13
2.3.2. Korrelation, Timeouts und Abbruch	13
2.3.3. Serviceadressierung und Routing	14
2.3.4. Bidirektionale Kommunikation und Streams	15
2.4. Ereignisse und Subscriptions	16
2.4.1. Abonnementmodell	16
2.4.2. Verteilung an mehrere Clients	16
2.4.3. Backpressure, Reihenfolge und Ressourcenfreigabe	17
2.5. Architektur- und Technologiegrundlagen	18
2.5.1. Plugin-Struktur und stabile Verträge	18
2.5.2. Adapter- und Proxy-Prinzip	18
2.5.3. gRPC und SignalR	19
2.5.4. Dependency Injection und Testbarkeit	20
2.6. Zusammenfassung	20

3. Anforderungsanalyse	21
3.1. Ausgangssituation	22
3.1.1. Analyse der bestehenden Kommunikationsarchitektur	22
3.1.2. Grenzen der bisherigen Serviceanbindung	22
3.2. Zielsystem und Rahmenbedingungen	22
3.2.1. Beteiligte Akteure und Komponenten	22
3.2.2. Bestehende Client- und WebAPI-Grenze	22
3.2.3. Minimaler Funktionsumfang	22
3.3. Funktionale Anforderungen	22
3.3.1. Registrierung und eindeutige Identifikation	22
3.3.2. Verwaltung von Integrationen und Devices	22
3.3.3. Heartbeat, Reconnect und Deregistrierung	22
3.3.4. Routing von Client-Aufträgen	22
3.3.5. SmartProxy und gRPC-Servicevertrag	22
3.3.6. Bereitstellung von TreeNode-Funktionen	22
3.3.7. Abonnieren von Wertänderungen	22
3.4. Nicht-funktionale Anforderungen	22
3.4.1. Wiederverwendbarkeit	22
3.4.2. Erweiterbarkeit	22
3.4.3. Zuverlässigkeit	22
3.4.4. Nebenläufigkeit	22
3.4.5. Performance	22
3.5. Abgrenzung	22
3.6. Zusammenfassung der Anforderungen	22
4. Konzeption	23
4.1. Ziel des Device-Integration-Plugins	24
4.2. Architekturübersicht	24
4.3. Konzeption der wiederverwendbaren Routing-Infrastruktur	24
4.4. Registrierung und Lebenszyklusverwaltung	24
4.4.1. Integration Registry	24
4.4.2. Lease Monitor	24
4.4.3. Timeout, Reconnect und Deregistrierung	24
4.5. Multi-Service-Routing	24
4.5.1. Serviceidentifikation und Routenauflösung	24
4.5.2. Job-Korrelation	24
4.5.3. Fehler- und Timeoutbehandlung	24
4.6. Contracts und Job-Envelopes	24
4.7. Servicevertrag und SmartProxy	24
4.7.1. IDeviceIntegrationService	24

4.7.2. DeviceIntegrationServiceProxy	24
4.7.3. Code-first gRPC	24
4.8. TreeNode-Adapter	24
4.9. Subscription-Konzept	24
4.10. Projekt- und Paketstruktur	24
4.11. Einbindung in zenLAB	24
4.12. Begründung zentraler Entwurfsentscheidungen	24
5. Implementierung	25
5.1. Technologische Rahmenbedingungen	26
5.2. Projektstruktur	26
5.3. Implementierung der Routing-Grundlage	26
5.4. Implementierung der Job-, Ergebnis- und Stream-Contracts	26
5.5. Implementierung des Multi-Service-Routings	26
5.6. Implementierung der Registrierung und Lebenszyklusverwaltung	26
5.7. Implementierung des gRPC-Servicevertrags und SmartProxys	26
5.8. Implementierung des Integration Clients	26
5.9. Implementierung des TreeNode-Adapters	26
5.10. Implementierung von Subscriptions	26
5.11. Einbindung in zenLAB	26
5.12. Anbindung eines Referenz-DeviceAgents	26
5.13. Herausforderungen während der Umsetzung	26
6. Evaluation	27
6.1. Ziel der Evaluation	28
6.2. Versuchsaufbau und Evaluationskriterien	28
6.3. Überprüfung der funktionalen Anforderungen	28
6.3.1. Registrierung und eindeutige Adressierung	28
6.3.2. Routing mit mehreren Integrationen und Clients	28
6.3.3. Heartbeat, Timeout und Reconnect	28
6.3.4. TreeNode-Aufrufe und Subscriptions	28
6.3.5. Kompatibilität der bestehenden Client- und WebAPI-Grenze	28
6.4. Bewertung der Wiederverwendbarkeit und Erweiterbarkeit	28
6.5. Messung des Routing-Overheads	28
6.6. Bewertung anhand des Referenz-DeviceAgents	28
6.7. Diskussion der Ergebnisse	28
6.8. Grenzen der Evaluation	28
7. Fazit und Ausblick	29
7.1. Zusammenfassung der Ergebnisse	29
7.2. Beantwortung der Forschungsfragen	29

7.3. Bewertung der Zielerreichung	29
7.4. Limitationen der Arbeit	29
7.5. Ausblick auf mögliche Erweiterungen	29
A. Supplemental Information	30
Abbildungsverzeichnis	32
Tabellenverzeichnis	33
List of Listings	34
Literaturverzeichnis	35
Glossar	37

Vorwort

Im Rahmen meines Verbundstudiums bei der infoteam Software AG konnte ich bereits vor Beginn meiner Bachelorarbeit in der Abteilung „Life Science“ wertvolle Einblicke in die Entwicklung von Komponenten für zenLAB gewinnen. Dabei mein besonderes Interesse an der Weiterentwicklung dieser Softwarelösung entstanden. Da die Anbindung von Geräten in der Medizintechnik eine zentrale und zugleich anspruchsvolle Aufgabe darstellt, beschäftigt sich diese Bachelorarbeit mit der Erweiterung der bestehenden Software um ein Device-Integration-Plugin. Ziel der Arbeit ist es, eine technische Grundlage zu schaffen, durch die neue Geräte künftig einfacher, flexibler und strukturierter in zenLAB integriert werden können.

An dieser Stelle möchte ich mich herzlich bei Jonas Krammer und Dr. Melanie Kahl für die Betreuung im Unternehmen, die fachliche Unterstützung sowie das hilfreiche Feedback während der Erstellung meiner Bachelorarbeit bedanken. Mein Dank gilt außerdem dem gesamten zenLAB Team für die interessanten Gespräche und den konstruktiven Austausch zur Entwicklung und Integration des Device-Integration-Plugins. Insbesondere durch gemeinsame Diskussionen konnten unterschiedliche Perspektiven eingebracht, Herausforderungen frühzeitig erkannt und mögliche Lösungsansätze weiterentwickelt werden.

Darüber hinaus danke ich Fabio Starke für seine Überarbeitungsvorschläge zur schriftlichen Ausarbeitung. Ebenso möchte ich mich bei Prof. Dr. Matthias Teßmann von der Technischen Hochschule Nürnberg für die Betreuung meiner Bachelorarbeit, die Beantwortung meiner Fragen sowie das wertvolle Feedback bedanken.

In der vorliegenden Arbeit werden Eigenbegriffe, Markennamen von Unternehmen oder staatlichen Einrichtungen sowie Begriffe aus dem Glossar bei ihrer ersten Nennung kursiv hervorgehoben. Codeausschnitte, beispielsweise Klassen- oder Methodennamen, werden im Fließtext farblich gekennzeichnet. Aus Gründen der besseren Lesbarkeit wird in dieser Arbeit das generische Maskulinum verwendet. Dieses ist geschlechtsneutral zu verstehen und bezieht sich gleichermaßen auf Personen jeglichen Geschlechts.

Kapitel 1.

Einleitung

Laborprozesse werden zunehmend digitalisiert, vernetzt und automatisiert. Geräte, Softwaredienste und Benutzeranwendungen sollen dabei nicht mehr isoliert arbeiten, sondern gemeinsam in durchgängige Abläufe eingebunden werden. Diese Entwicklung wird häufig unter dem Begriff Labor 4.0 zusammengefasst. Sie verfolgt das Ziel, Laborumgebungen flexibler, transparenter und effizienter zu gestalten [1]. Eine wesentliche Voraussetzung dafür ist, dass unterschiedliche Laborgeräte zuverlässig in übergeordnete Softwaresysteme integriert und von mehreren Anwendungen genutzt werden können.

Die praktische Umsetzung wird dadurch erschwert, dass Laborgeräte unterschiedliche Kommunikationsschnittstellen, Protokolle und Funktionsmodelle besitzen. Übergeordnete Anwendungen sollten diese technischen Besonderheiten jedoch nicht für jedes Gerät erneut berücksichtigen müssen. Benötigt wird daher eine vermittelnde Softwareebene, welche gerätespezifische Unterschiede kapselt und die verfügbaren Gerätefunktionen in einer einheitlichen Form zugänglich macht. Auch bestehende Ansätze zur Laborautomation zeigen, dass die Erkennung und zentrale Verwaltung verfügbarer Geräte und Dienste eine wichtige Grundlage für automatisierte Arbeitsabläufe darstellt [2].

Die vorliegende Arbeit entsteht in Zusammenarbeit mit der infoteam Software AG. Das Softwareunternehmen entwickelt kundenspezifische Lösungen unter anderem für die Bereiche Industry, Infrastructure, Life Science und Public Service. Ein Schwerpunkt liegt auf Software für regulierte Medizin- und Laborprodukte [3]. Im Bereich Life Science bietet infoteam mit zenLAB ein Middleware-Framework für vernetzte digitale Labore an [4].

zenLAB bildet den Anwendungskontext dieser Arbeit. Innerhalb der Middleware übernehmen sogenannte DeviceAgents die Kommunikation mit den konkreten Geräten. Sie kapseln hersteller- und protokollspezifische Besonderheiten und stellen Gerätefunktionen über ein gemeinsames TreeNode-Modell bereit. Dadurch können unterschiedliche Geräte grundsätzlich auf einheitliche Weise angesprochen werden. Mit einer wachsenden Zahl unabhängig betriebener DeviceAgents entsteht jedoch eine weitere Aufgabe: Die verfügbaren Integrationen und Geräte müssen zentral bekannt sein, ihr Zustand muss

überwacht werden und Zugriffe verschiedener Clients müssen zuverlässig an das jeweils zuständige Ziel vermittelt werden.

An diesem Punkt setzt die vorliegende Arbeit an. Ihr Gegenstand ist nicht nur die Weiterleitung einzelner Aufträge, sondern die Konzeption und prototypische Implementierung eines vollständigen Device-Integration-Plugins für zenLAB. Das Plugin soll die zentrale Verbindung zwischen Clientanwendungen und den bestehenden DeviceAgents bilden. Dazu gehören die Registrierung von Integrationen, die Verwaltung ihrer Geräte, die Überwachung ihres Lebenszyklus, die Vermittlung von Gerätezugriffen und die Weitergabe abonmierter Änderungen. Die gerätespezifische Kommunikation verbleibt weiterhin in den DeviceAgents.

1.1. Problemstellung

Die bestehende zenLAB-Architektur trennt die gerätespezifische Kommunikation bereits von den aufrufenden Anwendungen. Ein DeviceAgent kennt das angeschlossene Gerät und stellt dessen Funktionen bereit, während Clients über eine gemeinsame Schnittstelle auf diese Funktionen zugreifen. Diese Trennung erleichtert die Anbindung heterogener Geräte. Sie beantwortet jedoch noch nicht vollständig, wie mehrere zur Laufzeit hinzukommende Integrationen gemeinsam verwaltet und gleichzeitig von mehreren Clients genutzt werden können.

Ohne eine zentrale Integrationskomponente fehlt eine aktuelle Gesamtsicht auf die verfügbaren Integrationen und deren Geräte. Beim Hinzufügen, Neustarten oder Ausfall einer Integration muss nachvollziehbar bleiben, welche Geräte erreichbar sind. Eine erneute Verbindung darf nicht mit veralteten Zuständen der vorherigen Verbindung vermischt werden. Ebenso müssen gleichzeitige Clientzugriffe eindeutig dem richtigen Gerät zugeordnet und Ergebnisse an den jeweils auslösenden Aufruf zurückgegeben werden. Für abonnierte Geräteänderungen ist darüber hinaus eine dauerhafte und kontrollierbare Weitergabe an interessierte Clients erforderlich.

Die Herausforderung umfasst damit mehrere zusammenhängende Verantwortlichkeiten. Erstens müssen Integrationen eindeutig registriert und ihre Gerätelisten aktuell gehalten werden. Zweitens muss der Zustand einer Integration überwacht werden, damit Ausfälle erkannt und zugehörige Ressourcen vollständig bereinigt werden können. Drittens müssen Gerätezugriffe, Antworten und Ereignisse auch bei parallelen Abläufen korrekt zugeordnet werden. Schließlich soll sich die Lösung in die bestehende zenLAB-Architektur einfügen, ohne die etablierten Clientzugriffe oder die gerätespezifischen Implementierungen der DeviceAgents grundlegend zu verändern.

Hierfür soll ein Device-Integration-Plugin als zentrale Vermittlungs- und Verwaltungskomponente entwickelt werden. Das Plugin führt Informationen über Integrationen und Geräte zusammen, stellt ihre Funktionen für Clients bereit und behandelt Änderungen während der Laufzeit. Die hierfür benötigten Mechanismen sind nicht als isolierte Routinglösung zu betrachten, sondern als Bestandteile einer übergeordneten Geräteintegration. Registrierung, Lebenszyklusverwaltung, Gerätezuordnung, Auftragsvermittlung und Subscriptions müssen deshalb gemeinsam konzipiert und aufeinander abgestimmt werden.

1.2. Forschungsfragen

Aus der Problemstellung ergibt sich die übergeordnete Frage, wie ein Device-Integration-Plugin gestaltet werden kann, das mehrere heterogene und dynamisch verfügbare Geräteintegrationen zuverlässig in zenLAB einbindet. Die Arbeit folgt dabei den drei aufeinander aufbauenden Schritten Anforderungsanalyse, Entwicklung und Evaluation.

Im Rahmen der Arbeit werden folgende Forschungsfragen untersucht:

1. Welche funktionalen und nicht-funktionalen Anforderungen ergeben sich für ein Device-Integration-Plugin zur zuverlässigen Einbindung dynamischer Integrationen und ihrer Geräte in zenLAB?
2. Wie lässt sich ein solches Plugin auf Grundlage dieser Anforderungen konzipieren, prototypisch implementieren und in die bestehende Client- und DeviceAgent-Architektur einfügen?
3. Inwieweit erfüllt der entwickelte Prototyp die ermittelten Anforderungen, insbesondere hinsichtlich Funktionalität, Zuverlässigkeit, Integrationsaufwand und Performance?

Die erste Forschungsfrage ermittelt, welche Aufgaben und Qualitätsmerkmale das Plugin erfüllen muss. Betrachtet werden insbesondere die Registrierung mehrerer Integrationen, die Verwaltung und Aktualisierung ihrer Geräte, die Überwachung der Erreichbarkeit, der Umgang mit Abmeldung und erneuter Verbindung sowie die Nutzung durch mehrere Clients. Hinzu kommen Anforderungen an Fehlerbehandlung, Nebenläufigkeit, Erweiterbarkeit und die Einbindung in die vorhandene Systemlandschaft.

Die zweite Forschungsfrage überführt diese Anforderungen in eine Architektur und einen lauffähigen Prototyp. Im Mittelpunkt stehen die Verantwortlichkeiten und Schnittstellen

des Device-Integration-Plugins sowie das Zusammenspiel von Geräteverwaltung, Lebenszyklusüberwachung, Auftragsvermittlung und Subscriptions. Die technische Ausgestaltung wird aus den Anforderungen abgeleitet und in die bestehende zenLAB-Architektur integriert.

Die dritte Forschungsfrage bewertet die entwickelte Lösung anhand der zuvor festgelegten Anforderungen. Untersucht werden sowohl reguläre Abläufe mit mehreren Integrationen und Clients als auch Fehlerfälle wie doppelte Kennungen, nicht erreichbare Integrationen, Verbindungsabbrüche und erneute Registrierungen. Zusätzlich werden der notwendige Anpassungsaufwand für einen bestehenden DeviceAgent und der durch die zusätzliche Vermittlung entstehende Kommunikationsaufwand betrachtet.

1.3. Zielsetzung und Abgrenzung

Ziel der Arbeit ist ein prototypisch funktionsfähiges Device-Integration-Plugin, das mehrere Integrationen und die von ihnen bereitgestellten Geräte zentral in zenLAB verfügbar macht. Der Eigenbeitrag umfasst die Ermittlung der Anforderungen, den Entwurf der Plugin-Architektur, die Implementierung der zentralen Komponenten und die anschließende Evaluation. Das Routing von Aufträgen ist dabei ein notwendiger Teil der Lösung, bildet jedoch nicht allein den Gegenstand der Arbeit.

Der Prototyp soll folgende Kernfunktionen bereitstellen:

- mehrere Integrationen eindeutig registrieren und verwalten,
- die jeweils bereitgestellten Geräte erfassen und bei Änderungen aktualisieren,
- die Erreichbarkeit registrierter Integrationen überwachen,
- Abmeldung, Ausfall und erneute Verbindung konsistent behandeln,
- Gerätezugriffe gezielt an die zuständige Integration vermitteln,
- parallele Aufträge und Ergebnisse zuverlässig zuordnen,
- ausgewählte Geräteänderungen abonnieren und an Clients weitergeben sowie
- mindestens einen bestehenden DeviceAgent mit begrenztem Anpassungsaufwand anbinden.

Die Umsetzung wird mit mehreren gleichzeitig registrierten Integrationen und parallelen Clientzugriffen überprüft. Automatisierte Tests sollen sowohl die regulären Anwendungsfälle als auch relevante Fehler- und Nebenläufigkeitssituationen reproduzierbar abdecken. Ergänzend wird untersucht, welchen zusätzlichen Kommunikationsaufwand das Plugin

verursacht und welche Anpassungen für die Anbindung eines bestehenden DeviceAgents notwendig sind.

Nicht Ziel der Arbeit ist es, die vorhandenen DeviceAgents oder deren gerätespezifische Kommunikation zu ersetzen. Auch die bestehende öffentliche Clientschnittstelle soll nicht grundlegend verändert werden. Eine produktionsreife Migration sämtlicher DeviceAgents, eine persistierte oder verteilte Geräteverwaltung, umfassende Sicherheits- und Berechtigungskonzepte sowie eine vollständige produktive Gateway-Anbindung sind nicht Bestandteil des verpflichtenden Prototyps. Sie können jedoch als spätere Erweiterungen berücksichtigt werden.

1.4. Aufbau der Arbeit

Kapitel 2 erläutert die fachlichen und technischen Grundlagen. Dazu gehören die Rolle von zenLAB als Middleware, die Aufgaben von DeviceAgents und AgentManager, das TreeNode-Modell sowie grundlegende Konzepte für verteilte Kommunikation, Registrierung, Lebenszyklusüberwachung und die Vermittlung von Aufträgen und Ereignissen.

Kapitel 3 analysiert die bestehende Systemlandschaft und leitet daraus die funktionalen und nicht-funktionalen Anforderungen an das Device-Integration-Plugin ab. Zusätzlich werden der verbindliche Funktionsumfang des Prototyps und die Grenzen der Umsetzung festgelegt.

Kapitel 4 entwickelt auf Grundlage dieser Anforderungen das Lösungskonzept. Es beschreibt die Gesamtarchitektur des Plugins, die Verantwortlichkeiten der beteiligten Komponenten, die Verwaltung von Integrationen und Geräten, die Kommunikation mit Clients und DeviceAgents sowie den Umgang mit Subscriptions und Fehlerfällen.

Kapitel 5 dokumentiert die prototypische Implementierung und die Einbindung in zenLAB. Neben den zentralen Plugin-Komponenten werden die Anbindung eines bestehenden DeviceAgents und wesentliche Herausforderungen der Umsetzung dargestellt.

Kapitel 6 evaluiert den Prototyp anhand der festgelegten Anforderungen und definierter Testszenarien. Betrachtet werden Funktionalität, Fehlerverhalten, gleichzeitige Zugriffe, Integrationsaufwand und zusätzlicher Kommunikationsaufwand. Kapitel 7 fasst die Ergebnisse zusammen, beantwortet die Forschungsfragen und zeigt mögliche Weiterentwicklungen auf.

Kapitel 2.

Grundlagen

Dieses Kapitel beschreibt die fachlichen und technischen Grundlagen, die für die Entwicklung des Device-Integration-Plugins benötigt werden. Zunächst werden zenLAB und die bestehende Systemlandschaft mit DeviceAgents, AgentManager, TreeNode-Modell und Clientzugriff erläutert. Anschließend folgen grundlegende Konzepte zur dynamischen Geräteintegration, zur Kommunikation verteilter Komponenten und zur Verarbeitung von Ereignissen und Subscriptions. Den Abschluss bilden Architekturprinzipien und Kommunikationstechnologien, die für die spätere Konzeption relevant sind. Konkrete Entwurfsentscheidungen des zu entwickelnden Plugins werden erst in Kapitel 4 behandelt.

2.1. zenLAB und bestehende Systemlandschaft

2.1.1. Middleware- und Plugin-Architektur

zenLAB ist ein von der infoteam Software AG entwickeltes Middleware-Framework für vernetzte digitale Labore [4]. Eine Middleware bildet eine vermittelnde Softwareschicht zwischen technisch unterschiedlichen Komponenten. Im betrachteten Anwendungsfall trennt sie die gerätenahe Kommunikation von den Anwendungen, die Gerätefunktionen verwenden. Dadurch muss eine Clientanwendung weder das herstellerspezifische Protokoll noch die konkrete Verbindung eines Laborgeräts kennen.

Die Kapselung technischer Unterschiede ist für Laborumgebungen besonders relevant. Geräte verschiedener Hersteller können ihre Funktionen über serielle Verbindungen, Netzwerkprotokolle, proprietäre Bibliotheken oder standardisierte Schnittstellen bereitstellen. Ohne eine gemeinsame Vermittlungsschicht müsste jede übergeordnete Anwendung diese Unterschiede selbst behandeln. zenLAB stellt deshalb gemeinsame Strukturen und Dienste bereit, über die Gerätefunktionen in übergeordneten Anwendungen und automatisierten Abläufen verwendet werden können.

Die Architektur von zenLAB ist modular aufgebaut. Abgrenzbare Funktionen werden in Plugins gekapselt und durch einen Host als Dienste bereitgestellt. Die Interaktion zwischen Komponenten erfolgt über definierte Schnittstellen und gemeinsam verwendete Datentypen. Verträge und Implementierungen können dadurch getrennt entwickelt werden. Ein aufrufendes Modul ist nicht von einer konkreten Implementierung abhängig, solange der vereinbarte Vertrag eingehalten wird.

Eine solche Plugin-Architektur erleichtert die Erweiterung des Gesamtsystems. Neue Funktionen können als zusätzliche Module eingebunden werden, ohne sämtliche bestehenden Komponenten anzupassen. Gleichzeitig entstehen klare Anforderungen an die Verantwortlichkeiten und Abhängigkeiten der einzelnen Plugins. Ein Plugin sollte einen fachlich zusammengehörenden Funktionsbereich abdecken und nur über stabile Grenzen mit seiner Umgebung kommunizieren.

Für die vorliegende Arbeit bildet zenLAB den Laufzeit- und Integrationskontext. Das Device-Integration-Plugin soll innerhalb dieser modularen Architektur die zentrale Verwaltung und Vermittlung dynamisch verfügbarer Geräteintegrationen übernehmen. Es ergänzt damit die vorhandenen Bausteine, ersetzt jedoch weder die gerätespezifische Kommunikation noch die bereits verwendeten Clientzugriffe.

2.1.2. DeviceAgents und AgentManager

Ein DeviceAgent bildet die gerätenahe Integrationsschicht. Er kennt das Protokoll und die technischen Eigenschaften eines angebotenen Geräts und übersetzt zwischen dessen Schnittstelle und den von zenLAB erwarteten Funktionen. Zu seinen Aufgaben können der Aufbau einer Geräteverbindung, das Lesen und Schreiben von Werten, das Ausführen von Befehlen sowie die Verarbeitung spontan eintreffender Geräteereignisse gehören.

Die gerätespezifische Logik verbleibt bewusst im jeweiligen DeviceAgent. Nur dort ist bekannt, wie ein konkretes Gerät adressiert wird, welche Datentypen es verwendet und wie Fehler des Geräteprotokolls zu interpretieren sind. Diese Abgrenzung ermöglicht es, Geräte unterschiedlicher Hersteller über eine gemeinsame übergeordnete Struktur anzusprechen, ohne ihre technischen Besonderheiten zu verlieren.

DeviceAgents können in eigenen Prozessen ausgeführt werden. Die Prozessgrenze erhöht die Isolation, weil ein Fehler in einer gerätespezifischen Implementierung nicht unmittelbar im zenLAB-Host auftreten muss. Gleichzeitig können über diese Grenze keine direkten Objektreferenzen verwendet werden. Aufrufe, Parameter, Ergebnisse und Ereignisse müssen daher durch übertragbare Nachrichten beschrieben werden.

Der AgentManager übernimmt die administrative Verwaltung der DeviceAgents. Dazu gehören insbesondere deren Konfiguration sowie das Starten, Überwachen und Beenden

der zugehörigen Prozesse. Er kann erkennen, ob ein Prozess ausgeführt wird, und auf einen unerwarteten Prozessabbruch reagieren. Damit behandelt der AgentManager in erster Linie den Lebenszyklus ausführbarer Instanzen.

Davon ist der Kommunikationslebenszyklus einer Integration zu unterscheiden. Ein gestarteter Prozess ist nicht automatisch beim späteren Device-Integration-Plugin registriert und für Clients erreichbar. Umgekehrt kann eine Kommunikationsverbindung unterbrochen sein, obwohl der Prozess weiterhin läuft. Der AgentManager beantwortet daher die Frage, ob ein DeviceAgent-Prozess ausgeführt wird. Das neue Plugin muss dagegen feststellen, welche Integrationen sich aktuell angemeldet haben, welche Geräte sie bereitstellen und ob ihre Kommunikationsverbindung noch gültig ist.

Diese Trennung verhindert eine Vermischung unterschiedlicher Verantwortlichkeiten. Der AgentManager bleibt für Prozessstart, Prozessüberwachung und Konfiguration zuständig. Das Device-Integration-Plugin übernimmt die logische Verfügbarkeit und die Kommunikation der registrierten Integrationen. Beide Komponenten ergänzen einander, ohne denselben Lebenszyklus doppelt abzubilden.

2.1.3. TreeNode-Modell

Damit Clients Geräte unterschiedlicher Hersteller einheitlich verwenden können, bildet zenLAB deren Funktionen als Baumstruktur ab. Ein Gerät besitzt einen Wurzelknoten, unter dem weitere Knoten hierarchisch angeordnet sind. Jeder Knoten ist über einen Pfad innerhalb dieses Baums erreichbar. Auf diese Weise lassen sich Gerätefunktionen unabhängig von der zugrunde liegenden Hardware mit einer einheitlichen Adressierungsform beschreiben.

Ein TreeNode kann unterschiedliche Rollen besitzen. Ein Knoten kann beispielsweise einen lesbaren oder veränderbaren Wert, eine aufrufbare Methode oder einen strukturellen Teil des Gerätemodells darstellen. Untergeordnete Knoten gliedern zusammengehörende Funktionen. Die Baumstruktur ermöglicht es einem Client, das Modell eines Geräts zu erkunden und anschließend gezielt auf einzelne Funktionen zuzugreifen.

Im bestehenden System fasst die Schnittstelle `ITreeNodeBasedDeviceAgent` die zentralen TreeNode-Operationen zusammen. Dazu gehören das Ermitteln von Knoteninformationen, das Lesen und Setzen von Werten, das Aufrufen von Methoden sowie das Anlegen und Beenden von Abonnements. Der konkrete DeviceAgent übersetzt diese generischen Operationen in die Kommunikation mit dem jeweiligen Laborgerät.

Neben einmaligen Aufrufen unterstützt das TreeNode-Modell Ereignisse. Ein Client kann sich beispielsweise für Änderungen eines Werts oder der Knotenstruktur interessieren. Der DeviceAgent meldet solche Änderungen, ohne dass der Client den Zustand

fortlaufend abfragen muss. Diese ereignisbasierte Kommunikation ist für das Device-Integration-Plugin ebenso relevant wie normale Anfrage-Antwort-Aufrufe.

Bestehende DeviceAgents sollen unverändert weiterverwendet werden. Dafür ist später eine technische Anpassungsschicht erforderlich. Sie bildet übertragene Aufträge auf die vorhandenen TreeNode-Operationen ab. Die fachliche Bedeutung der Gerätefunktionen und die eigentliche Hardwarekommunikation bleiben davon unberührt.

2.1.4. Bestehende Client- und WebAPI-Grenze

Clientanwendungen verwenden den `TreeNodeBasedClient`, um auf `TreeNode`-basierte Geräte zuzugreifen. Die Bibliothek kapselt die technischen Details der Kommunikation mit der `TreeNodeBased.WebAPI` und stellt typisierte Clientfunktionen bereit. Dadurch arbeiten die Anwendungen mit Geräte- und `TreeNode`-Operationen, ohne die zugrunde liegenden Webaufrufe selbst erzeugen zu müssen.

Auftragsbezogene Operationen werden über HTTP an die WebAPI übertragen. Dazu zählen beispielsweise das Lesen eines Werts, das Setzen eines Werts oder das Ausführen einer Methode. Die WebAPI nimmt die Anfrage entgegen, löst die angegebenen Geräte- und Knotenpfade auf und delegiert die eigentliche Geräteoperation an die interne Dienstschicht.

Für ereignisbasierte Informationen verwendet die bestehende Grenze ASP.NET Core SignalR. Über eine länger bestehende Verbindung können Wert- und Geräteänderungen an verbundene Clients weitergegeben werden. SignalR-Hubs ermöglichen dabei sowohl Aufrufe von Clients an den Server als auch Nachrichten des Servers an einzelne Clients, Gruppen oder alle verbundenen Clients [5].

`TreeNodeBasedClient` und `WebAPI` bilden eine bereits verwendete Systemgrenze. Ihre grundsätzliche Funktionsweise soll durch die Arbeit nicht verändert werden. Das Device-Integration-Plugin wird hinter dieser Grenze eingeordnet. Die WebAPI bleibt damit für webbezogene Aufgaben zuständig, während das neue Plugin die dahinterliegende Verwaltung und Vermittlung der Integrationen und Geräte übernimmt.

2.2. Dynamische Geräteintegration

2.2.1. Integrationen, Registrierungen und Geräte

Für die weitere Arbeit müssen die Begriffe Integration, Registrierung und Gerät voneinander unterschieden werden. Eine Integration bezeichnet eine eigenständig kommunizierende Softwarekomponente, die dem Plugin ein oder mehrere Geräte zur Verfügung

stellt. Ein DeviceAgent kann eine solche Integration bilden oder an eine entsprechende Laufzeitkomponente angebunden werden. Ein Gerät bezeichnet dagegen die fachliche Einheit, deren TreeNode-Funktionen von Clients verwendet werden.

Eine Integration besitzt eine logische Identität, über die sie systemweit unterschieden werden kann. Diese Identität soll auch dann stabil bleiben, wenn die Komponente neu gestartet wird. Eine konkrete Anmeldung am Plugin ist dagegen zeitlich begrenzt. Nach einem Neustart oder einer erneuten Verbindung entsteht eine neue Registrierung, obwohl die logische Integration dieselbe bleiben kann.

Die Trennung zwischen stabiler Integrationsidentität und konkreter Registrierung ist für die Konsistenz des Systems wesentlich. Andernfalls könnte eine verspätete Nachricht aus einer alten Verbindung irrtümlich als Nachricht der neuen Verbindung behandelt werden. Gleiches gilt für bereits gestartete Timeouts oder Aufräumoperationen. Sie dürfen ausschließlich den Zustand derjenigen Registrierung verändern, zu der sie gehören.

Auch Geräte benötigen eine eindeutige Identität. Ein Client soll ein Gerät unabhängig davon adressieren können, über welche technische Verbindung seine Integration gerade kommuniziert. Das Plugin muss deshalb eine Zuordnung zwischen Geräten, Integrationen und den jeweils aktuellen Registrierungen verwalten. Ändert sich die von einer Integration bereitgestellte Geräteliste, muss diese Zuordnung konsistent aktualisiert werden.

2.2.2. Registrierung und zentrale Geräteverwaltung

Dynamisch verfügbare Integrationen müssen dem Gesamtsystem mitteilen, dass sie erreichbar sind und welche Geräte sie bereitstellen. Dieser Vorgang wird als Registrierung bezeichnet. Dabei übermittelt eine Integration ihre Identität, beschreibende Metadaten und eine Liste der aktuell verfügbaren Geräte. Erst nach einer erfolgreichen Registrierung kann sie bei der Vermittlung von Clientzugriffen berücksichtigt werden.

Eine zentrale Registry verwaltet den aktuellen Zustand der angemeldeten Integrationen. Sie beantwortet unter anderem, ob eine Integration aktiv ist, welche Registrierung zu ihr gehört und welche Geräte über sie erreichbar sind. Damit bildet sie die Grundlage für die Geräteauflösung und für spätere Routingentscheidungen. Auch andere Ansätze zur Laborautomation nutzen eine zentrale Verwaltung verfügbarer Geräte und Dienste, um deren Funktionen für automatisierte Abläufe auffindbar zu machen [2].

Gerätelisten sind nicht notwendigerweise statisch. Eine Integration kann während ihrer Laufzeit weitere Geräte erkennen oder bisher verfügbare Geräte verlieren. Das Plugin benötigt deshalb ein definiertes Aktualisierungsmodell. Eine Möglichkeit ist die Übertragung vollständiger Bestandsaufnahmen. Der Empfänger kann einen solchen Snapshot atomar übernehmen und erkennt eindeutig, welche Geräte nach der Aktualisierung

vorhanden sind. Bei rein inkrementellen Meldungen muss dagegen zusätzlich sichergestellt werden, dass keine Änderung verloren geht oder in falscher Reihenfolge verarbeitet wird.

Die Registry ist eine gemeinsam verwendete Datenstruktur. Registrierungen, Geräteaktualisierungen, Clientzugriffe und Aufräumvorgänge können gleichzeitig auftreten. Zustandsänderungen müssen deshalb atomar oder durch geeignete Synchronisationsmechanismen geschützt werden. Ein Client darf beispielsweise kein Gerät erhalten, dessen Registrierung im selben Moment bereits entfernt wurde.

Die zentrale Geräteverwaltung erfüllt damit mehr als eine reine Verzeichnisfunktion. Sie bildet die aktuelle Sicht auf die verfügbaren Integrationen und Geräte, stellt konsistente Zuordnungen für Clientzugriffe bereit und dient als Ausgangspunkt für Lebenszyklusüberwachung, Kommunikation und Subscriptions.

2.2.3. Lebenszyklus, Lease und Reconnect

Eine erfolgreiche Registrierung belegt nur, dass eine Integration zu einem bestimmten Zeitpunkt erreichbar war. Prozesse können abstürzen, Netzwerkverbindungen können abbrechen und ein ordnungsgemäßer Abmeldeaufruf kann ausbleiben. Das Plugin benötigt daher einen Mechanismus, um veraltete Registrierungen auch ohne aktive Abmeldung zu erkennen.

Ein verbreiteter Ansatz ist ein zeitlich begrenztes Lease. Ein Lease gewährt einer Komponente einen Zustand oder ein Nutzungsrecht nur für einen festgelegten Zeitraum und muss regelmäßig erneuert werden. Das Konzept wurde als fehlertoleranter Mechanismus für verteilte Systeme beschrieben [6]. Bleibt die Erneuerung aus, kann der Empfänger nach Ablauf des Zeitraums davon ausgehen, dass die zugehörige Registrierung nicht mehr gültig ist.

Die Erneuerung kann durch eine eigene Statusmeldung oder durch eine regelmäßig übertragene vollständige Geräteliste erfolgen. Entscheidend ist, dass Intervall und Toleranz eindeutig festgelegt sind. Eine zu kurze Frist kann bereits bei vorübergehenden Verzögerungen zu einer fälschlichen Abmeldung führen. Eine zu lange Frist lässt nicht mehr erreichbare Geräte unnötig lange als verfügbar erscheinen.

Beim Ablauf oder bei der aktiven Abmeldung einer Registrierung müssen sämtliche zugehörigen Zustände bereinigt werden. Dazu gehören die Gerätezuordnungen, offene Aufträge, Kommunikationskanäle und aktive Subscriptions. Der Vorgang sollte idempotent sein. Mehrere gleichzeitig ausgelöste Aufräumvorgänge müssen zum gleichen Endzustand führen und dürfen keine bereits neu angelegte Registrierung entfernen.

Ein Reconnect erzeugt eine besondere Nebenläufigkeitssituation. Eine Integration kann sich mit ihrer stabilen Identität erneut anmelden, während Nachrichten oder Timeouts der alten Verbindung noch eintreffen. Deshalb müssen eingehende Nachrichten und Ausräumvorgänge zusätzlich der konkreten Registrierung zugeordnet werden. Nur Zustände der aktuell betroffenen Registrierung dürfen verändert werden.

2.3. Kommunikation verteilter Komponenten

2.3.1. Auftrags- und Ergebnisprinzip

Bei einem lokalen Methodenaufruf befinden sich Aufrufer und ausführende Komponente üblicherweise im selben Prozess. Über eine Prozess- oder Netzwerkgrenze ist dies nicht möglich. Die gewünschte Operation muss in eine übertragbare Nachricht überführt und auf der Gegenseite wieder einer ausführbaren Funktion zugeordnet werden.

Ein auftragsbasiertes Modell beschreibt eine solche Nachricht als Auftrag beziehungsweise Job. Der Auftrag enthält eine Kennung, die gewünschte Operation, die benötigten Parameter und gegebenenfalls Informationen über das Ziel. Parameter müssen in einem Format dargestellt werden, das vom verwendeten Transport übertragen und auf der Gegenseite rekonstruiert werden kann.

Nach der Bearbeitung erzeugt die ausführende Komponente ein Ergebnis. Dieses verweist auf den ursprünglichen Auftrag und enthält entweder einen Rückgabewert oder eine definierte Fehlerdarstellung. Ausnahmen können nicht als lokale Laufzeitobjekte über eine Prozessgrenze weitergereicht werden. Relevante Fehlerinformationen müssen deshalb in einen transportierbaren Datentyp überführt werden.

Das Auftrags- und Ergebnisprinzip entkoppelt die fachliche Operation vom Transport. Der Aufrufer muss nicht wissen, in welchem Prozess oder durch welche konkrete Klasse ein Auftrag ausgeführt wird. Beide Seiten benötigen lediglich gemeinsame Verträge für Operationen, Parameter, Ergebnisse und Fehler. Diese Verträge bilden eine stabile Grenze zwischen Client, Plugin und Integration.

2.3.2. Korrelation, Timeouts und Abbruch

Mehrere Clients können gleichzeitig Aufträge senden. Auch eine einzelne Integration kann mehrere Aufträge parallel bearbeiten. Ergebnisse treffen daher nicht zwingend in derselben Reihenfolge ein, in der die Aufträge versendet wurden. Für eine korrekte Zuordnung benötigt jeder Auftrag eine eindeutige Kennung.

Eine Korrelationskomponente merkt sich, welche wartende Operation zu welcher Auftragskennung gehört. Sobald ein passendes Ergebnis eintrifft, wird genau diese Operation abgeschlossen. Ergebnisse mit unbekannter oder bereits abgeschlossener Kennung dürfen nicht beliebig einem anderen Auftrag zugeordnet werden. Sie werden verworfen und können für Diagnosezwecke protokolliert werden.

Zusätzlich zur Auftragskennung ist in einem System mit mehreren Integrationen der Kontext des Ziels relevant. Dieselbe numerische Kennung könnte in unterschiedlichen Verbindungen vorkommen. Eine robuste Prüfung berücksichtigt daher auch die logische Integration und die konkrete Registrierung. Auf diese Weise lässt sich verhindern, dass ein verspätetes Ergebnis einer alten Verbindung einen neuen Auftrag abschließt.

Ein Timeout begrenzt die Wartezeit auf ein Ergebnis. Bleibt eine Antwort aus, wird die wartende Operation mit einem definierten Fehler beendet und ihr Korrelationszustand entfernt. Abbruchsignale erlauben es außerdem, Aufträge zu beenden, wenn ein Client seine Anfrage zurückzieht, eine Verbindung getrennt wird oder das Gesamtsystem herunterfährt.

Timeout und Abbruch beenden zunächst das lokale Warten. Die entfernte Verarbeitung kann zu diesem Zeitpunkt bereits begonnen haben. Die Contracts und die Fehlerbehandlung müssen deshalb berücksichtigen, dass ein Ergebnis noch verspätet eintreffen kann. Eine eindeutige Zustandsprüfung verhindert, dass ein solcher Eingang bereits abgeschlossene Abläufe nachträglich verändert.

2.3.3. Serviceadressierung und Routing

Routing bezeichnet die Auswahl des Empfängers, der einen Auftrag bearbeiten soll. In einem System mit nur einer festen Gegenstelle kann das Ziel implizit sein. Sobald mehrere Integrationen gleichzeitig aktiv sind, muss jeder Auftrag eine ausreichende Zielinformation enthalten oder über die angegebene Geräteidentität auf ein Ziel aufgelöst werden können.

Die Registry liefert die Zuordnung zwischen einem Gerät und der aktuell zuständigen Registrierung. Eine Routingkomponente verwendet diese Zuordnung, um den Auftrag an den passenden Kommunikationskanal zu übergeben. Ist das Gerät unbekannt, die Integration nicht aktiv oder die Registrierung inzwischen ersetzt worden, muss der Zugriff mit einem nachvollziehbaren Fehler beendet werden.

Beim Unicast-Routing wird ein Auftrag an genau ein Ziel übertragen. Dieses Verfahren eignet sich für gerätebezogene Operationen, weil ein konkretes Gerät zu einem bestimmten Zeitpunkt genau einer aktiven Integration zugeordnet sein sollte. Die Antwort kann unmittelbar mit dem versendeten Auftrag korreliert werden.

Ein Broadcast richtet einen Auftrag dagegen an mehrere Integrationen. Dadurch entstehen zusätzliche Anforderungen. Für jedes Ziel muss ein eigener Teilauftrag beziehungsweise Korrelationszustand geführt werden. Die Teilergebnisse können zu unterschiedlichen Zeitpunkten eintreffen, einzelne Ziele können fehlschlagen und ein Gesamttimeout muss den Abschluss der Aggregation bestimmen. Zudem ist festzulegen, ob ein Teilausfall das gesamte Ergebnis ungültig macht oder als Teilresultat ausgewiesen wird.

Routing ist damit eine notwendige Teilfunktion der Geräteintegration, jedoch nicht mit ihr gleichzusetzen. Erst durch die Verbindung mit Registry, Lebenszyklusverwaltung, Gerätezuordnung und Fehlerbehandlung kann zuverlässig entschieden werden, ob und wohin ein Auftrag übertragen werden darf.

2.3.4. Bidirektionale Kommunikation und Streams

Neben einzelnen Anfrage-Antwort-Aufrufen benötigt das Plugin länger bestehende Kommunikationswege. Eine registrierte Integration muss Aufträge empfangen und gleichzeitig Ergebnisse oder Ereignisse zurücksenden können. Eine bidirektionale Verbindung ermöglicht, dass beide Seiten unabhängig voneinander Nachrichten übertragen, ohne für jedes Element eine neue Verbindung aufzubauen.

Asynchrone Streams modellieren eine Folge von Elementen, deren Umfang und Ankunftszeit beim Start noch nicht vollständig bekannt sind. In C# können solche Folgen mit `IAsyncEnumerable<T>` beschrieben und mit `await foreach` verarbeitet werden. Dabei wird jedes Element verarbeitet, sobald es verfügbar ist, ohne während der gesamten Wartezeit einen Thread zu blockieren [7].

Für die interne Kommunikation verteilter .NET-Komponenten kann gRPC eingesetzt werden. Es unterstützt neben einzelnen Aufrufen auch Client-, Server- und bidirektionales Streaming. Beim code-first-Ansatz werden Service- und Nachrichtenverträge durch .NET-Schnittstellen und Datentypen beschrieben. Dadurch können Client und Server gemeinsame Verträge verwenden, ohne zusätzlich eine manuell gepflegte `.proto`-Datei als primäre Quelle zu benötigen [8].

Eine dauerhaft geöffnete Verbindung ist dennoch kein dauerhafter Verfügbarkeitsnachweis. Streamende, Transportfehler und Abbruchsignale müssen ausgewertet und mit dem Registrierungszustand verknüpft werden. Ebenso muss der Besitzer einer Verbindung eindeutig festgelegt sein, damit sie bei Abmeldung, Timeout oder Herunterfahren zuverlässig beendet und freigegeben wird.

2.4. Ereignisse und Subscriptions

2.4.1. Abonnementmodell

Nicht jede Geräteinformation wird durch einen Client aktiv abgefragt. Werte oder Strukturen können sich unabhängig von einem Clientaufruf verändern. Polling würde erfordern, dass Clients dieselben Werte in kurzen Abständen wiederholt lesen. Dies erzeugt unnötige Aufrufe und erkennt Änderungen nur mit einer vom Abfrageintervall abhängigen Verzögerung.

Ein Abonnement beschreibt das fortlaufende Interesse eines Clients an einer bestimmten Ereignisquelle. Im `TreeNode`-Kontext kann die Quelle durch das Gerät, einen Knotenpfad und die gewünschte Ereignisart bestimmt werden. Nach erfolgreicher Einrichtung werden passende Änderungen so lange übertragen, bis das Abonnement beendet wird oder eine beteiligte Verbindung ausfällt.

Der Lebenszyklus eines Abonnements unterscheidet sich von einem einmaligen Auftrag. Zwischen Einrichtung und Beendigung können viele Ereignisse eintreffen. Das System muss erkennen, welchem Client sie zuzustellen sind, wann ein Abonnement nicht mehr gültig ist und welche Ressourcen nach seinem Ende freigegeben werden müssen.

Subscriptions sind daher kein bloßer Sonderfall eines einzelnen Ergebnisses. Sie benötigen eigene Identitäten und Zustände sowie eine definierte Behandlung von Fehlern, Verbindungsabbrüchen und erneuter Registrierung. Ereignisse einer alten Registrierung dürfen ebenso wenig an Clients weitergegeben werden wie verspätete Antworten alter Aufträge.

2.4.2. Verteilung an mehrere Clients

Mehrere Clients können dieselbe Ereignisquelle abonnieren. Würde für jeden Client ein eigenständiges Abonnement bis zum `DeviceAgent` aufgebaut, könnten identische Geräteereignisse mehrfach erzeugt und übertragen werden. Eine gemeinsame vorgelagerte Subscription mit anschließender Verteilung reduziert diesen Aufwand.

Hierfür verwaltet das Plugin einen logischen Schlüssel, der die abonnierte Quelle eindeutig beschreibt. Beim ersten interessierten Client wird das vorgelagerte Abonnement eingerichtet. Weitere Clients werden als zusätzliche Empfänger derselben Quelle registriert. Eingehende Ereignisse werden anschließend an alle aktuell zugeordneten Clients verteilt.

Eine Referenzzählung kann bestimmen, wann das vorgelagerte Abonnement nicht mehr benötigt wird. Beendet ein Client sein Interesse, wird nur seine Zuordnung entfernt.

Erst wenn kein Client mehr verbleibt, kann auch die Subscription gegenüber der Integration aufgelöst werden. Neben der Anzahl müssen die konkreten Clientzuordnungen gespeichert werden, damit ein Verbindungsabbruch gezielt bereinigt werden kann.

Bei der Verteilung ist eine langsame Clientverbindung von anderen Empfängern zu isolieren. Andernfalls könnte ein einzelner Client die Verarbeitung aller weiteren Ereignisse verzögern. Geeignete Warteschlangen oder getrennte Verarbeitungspfade ermöglichen ein unabhängiges Fan-out. Dabei muss festgelegt werden, welche Reihenfolge pro Subscription garantiert wird.

2.4.3. Backpressure, Reihenfolge und Ressourcenfreigabe

Produziert eine Ereignisquelle dauerhaft mehr Elemente, als ein Empfänger verarbeiten kann, entsteht Backpressure. Ein unbegrenzter Puffer würde in diesem Fall kontinuierlich wachsen und langfristig den Speicherverbrauch des Systems erhöhen. Begrenzte Warteschlangen machen den maximalen Ressourcenbedarf kontrollierbar.

Ist ein Puffer gefüllt, benötigt das System eine definierte Strategie. Es kann die produzierende Seite vorübergehend warten lassen, neue Ereignisse ablehnen oder überholte Zustandsmeldungen ersetzen. Welche Variante fachlich zulässig ist, hängt von der Ereignisart ab. Bei einer Folge einzelner Befehlsresultate darf in der Regel kein Element verloren gehen. Bei häufigen Meldungen desselben aktuellen Zustands kann dagegen unter Umständen nur der neueste Wert relevant sein.

Auch die Reihenfolge der Ereignisse muss betrachtet werden. Für Änderungen derselben Quelle ist häufig eine geordnete Zustellung erforderlich. Zwischen voneinander unabhängigen Geräten oder Subscriptions kann eine parallele Verarbeitung dagegen sinnvoll sein. Die gewünschte Garantie muss vor der Wahl der Datenstrukturen und Verarbeitungsstrategie feststehen.

Abbruch und Ressourcenfreigabe gehören zum Subscription-Lebenszyklus. Endet eine Clientverbindung, läuft eine Registrierung ab oder wird das Plugin beendet, müssen zugehörige Streams, Warteschlangen und Abonnements geschlossen werden. Abbruchsignale sollten durch alle beteiligten Verarbeitungsschichten weitergegeben werden. Eine idempotente Bereinigung verhindert Fehler, wenn mehrere Ursachen nahezu gleichzeitig dasselbe Abonnement beenden.

2.5. Architektur- und Technologiegrundlagen

2.5.1. Plugin-Struktur und stabile Verträge

Eine Plugin-Architektur trennt einen stabilen Kern von ergänzbaren Funktionsbereichen. Die Interaktion erfolgt über definierte Erweiterungspunkte und Verträge. Solche Architekturen unterstützen eine unabhängige Entwicklung und Bereitstellung einzelner Komponenten, erfordern jedoch eine kontrollierte Verwaltung von Schnittstellen und Abhängigkeiten [9].

Verträge beschreiben, welche Operationen eine Komponente anbietet und welche Daten dabei ausgetauscht werden. Sie sollten nur Informationen enthalten, die für die Kommunikation tatsächlich erforderlich sind. Interne Implementierungsdetails gehören nicht in öffentliche Contracts, weil Änderungen daran sonst unnötig viele abhängige Projekte betreffen würden.

Für ein Device-Integration-Plugin sind mehrere fachliche Bereiche zu unterscheiden: die Kommunikation mit Clients, die Kommunikation mit Integrationen, die Verwaltung von Registrierungen und Geräten sowie die Verteilung von Ereignissen. Ob diese Bereiche in getrennten Projekten umgesetzt werden, ist eine spätere Entwurfsentscheidung. Bereits auf Grundlagenebene ist jedoch wichtig, dass ihre Verantwortlichkeiten nicht in einer einzigen allzuständigen Komponente vermischt werden.

Auch transportbezogene und fachliche Datentypen sollten bewusst getrennt werden. Allgemeine Auftrags-, Ergebnis- und Routinginformationen können unabhängig von einer konkreten TreeNode-Operation sein. TreeNode-spezifische Contracts beschreiben dagegen Gerätepfade, Werte, Methodenparameter und Ereignisse. Diese Trennung unterstützt Wiederverwendung und begrenzt Abhängigkeiten.

2.5.2. Adapter- und Proxy-Prinzip

Das Adapter-Entwurfsmuster überführt die Schnittstelle einer bestehenden Komponente in eine Schnittstelle, die von einem neuen Kontext erwartet wird. Dadurch kann vorhandene Funktionalität weiterverwendet werden, ohne die bestehende Implementierung vollständig anzupassen [10]. Für die Arbeit ist dieses Prinzip auf der Seite der Device-Agents relevant.

Ein Integrationsadapter kann transportierte TreeNode-Aufträge entgegennehmen und auf die vorhandene DeviceAgent-Schnittstelle abbilden. Nach der Ausführung übersetzt

er Ergebnisse, Fehler und Ereignisse zurück in die gemeinsamen Kommunikationsverträge. Er besitzt dabei keine gerätespezifische Fachlogik. Diese verbleibt im angebundenen DeviceAgent.

Das Proxy-Prinzip stellt einem Aufrufer ein lokales Objekt zur Verfügung, das dieselbe fachliche Schnittstelle wie ein entfernter Dienst anbietet. Der Proxy verbirgt Verbindungsaufbau, Serialisierung und Remote-Aufruf. Für den aufrufenden Teil des Systems bleibt dadurch die fachliche Dienstgrenze sichtbar, während Transportdetails gekapselt werden.

In zenLAB wird dieses Prinzip in Form von SmartProxys verwendet. Für das Grundlagenverständnis ist entscheidend, dass Proxy und serverseitiger Dienst denselben Vertrag verwenden können. Welche konkreten Serviceverträge und Proxyklassen das Device-Integration-Plugin benötigt, wird erst im Architekturkonzept festgelegt.

2.5.3. gRPC und SignalR

gRPC und SignalR erfüllen im betrachteten System unterschiedliche Aufgaben. gRPC eignet sich für klar definierte Dienstgrenzen zwischen internen Komponenten. Es unterstützt typisierte Anfrage-Antwort-Aufrufe und verschiedene Streamingformen. Code-first gRPC ermöglicht es, Dienstverträge mit .NET-Schnittstellen und Datentypen auszudrücken [8].

SignalR stellt eine Hub-basierte Echtzeitkommunikation für verbundene Clients bereit. Der Server kann Nachrichten gezielt an einzelne Verbindungen oder Gruppen senden. Darüber hinaus unterstützt SignalR Streaming vom Client zum Server und vom Server zum Client. Elemente können übertragen werden, sobald sie verfügbar sind, anstatt zunächst eine vollständige Ergebnismenge zu sammeln [5, 11].

An der bestehenden äußeren Systemgrenze verwendet die TreeNodeBased.WebAPI SignalR für Benachrichtigungen an Clientanwendungen. Für die interne Kommunikation zwischen Plugin und weiteren .NET-Komponenten kann gRPC verwendet werden. Die Technologien ersetzen einander daher nicht, sondern bedienen verschiedene Kommunikationsgrenzen.

Unabhängig von der Transporttechnologie benötigt die Anwendung eigene Regeln für Identität, Lebenszyklus, Korrelation und Fehlerbehandlung. Ein Transport kann einen Verbindungsabbruch melden, entscheidet aber nicht selbst, welche Registrierung dadurch ungültig wird oder welche Geräte und Subscriptions zu entfernen sind. Diese Entscheidungen gehören zur Anwendungslogik des Plugins.

2.5.4. Dependency Injection und Testbarkeit

Dependency Injection bezeichnet die Bereitstellung von Abhängigkeiten von außen, anstatt sie innerhalb einer Klasse fest zu erzeugen. Dadurch hängt eine Komponente von einem Vertrag und nicht zwingend von einer konkreten Implementierung ab. In .NET sind Dependency Injection und ein zugehöriger Servicecontainer Bestandteil des Frameworks [12].

Für das Device-Integration-Plugin betrifft dies beispielsweise Registry, Lebenszyklusüberwachung, Routing, Subscription-Verwaltung und Transportdienste. Werden diese Abhängigkeiten über Konstruktoren bereitgestellt, bleiben Verantwortlichkeiten sichtbar. Gleichzeitig können in automatisierten Tests kontrollierte Testimplementierungen eingesetzt werden.

Testbarkeit ist bei nebenläufigen und zeitabhängigen Abläufen besonders wichtig. Eine abstrahierte Zeitquelle ermöglicht es beispielsweise, Lease-Abläufe und Timeouts deterministisch zu prüfen. Kontrollierbare Kommunikationskanäle erlauben Tests für verspätete Ergebnisse, Streamabbrüche und parallele Registrierungen, ohne von realen Netzwerklaufzeiten abhängig zu sein.

Dependency Injection allein garantiert noch keine gute Struktur. Zu viele Abhängigkeiten können darauf hinweisen, dass eine Klasse mehrere Verantwortlichkeiten übernimmt. Kleine, klar benannte Komponenten und explizite Zuständigkeiten erleichtern sowohl die Implementierung als auch die spätere Evaluation der einzelnen Anforderungen.

2.6. Zusammenfassung

Das bestehende zenLAB-System stellt mit DeviceAgents, AgentManager, TreeNode-Modell sowie Client- und WebAPI-Grenze wesentliche Bausteine für die geräteunabhängige Kommunikation bereit. Für die dynamische Einbindung mehrerer Integrationen werden darüber hinaus eine zentrale Sicht auf Registrierungen und Geräte, eine eigenständige Lebenszyklusüberwachung und eine zuverlässige Vermittlung von Aufträgen und Ereignissen benötigt.

Die beschriebenen Grundlagen zeigen, dass das Device-Integration-Plugin mehrere zusammenhängende Aufgaben vereint. Routing ist eine davon, steht jedoch in direkter Abhängigkeit zu Registrierung, Geräteverwaltung, Korrelation, Subscriptions und Ressourcenbereinigung. Auf dieser Grundlage werden im folgenden Kapitel die konkreten Anforderungen an das Plugin hergeleitet.

Kapitel 3.

Anforderungsanalyse

3.1. Ausgangssituation

3.1.1. Analyse der bestehenden Kommunikationsarchitektur

3.1.2. Grenzen der bisherigen Serviceanbindung

3.2. Zielsystem und Rahmenbedingungen

3.2.1. Beteiligte Akteure und Komponenten

3.2.2. Bestehende Client- und WebAPI-Grenze

3.2.3. Minimaler Funktionsumfang

3.3. Funktionale Anforderungen

3.3.1. Registrierung und eindeutige Identifikation

3.3.2. Verwaltung von Integrationen und Devices

3.3.3. Heartbeat, Reconnect und Deregistrierung

3.3.4. Routing von Client-Aufträgen

3.3.5. SmartProxy und gRPC-Servicevertrag

3.3.6. Bereitstellung von TreeNode-Funktionen

3.3.7. Abonnieren von Wertänderungen

3.4. Nicht-funktionale Anforderungen

3.4.1. Wiederverwendbarkeit

3.4.2. Erweiterbarkeit

3.4.3. Zuverlässigkeit

Kapitel 4.

Konzeption

4.1. Ziel des Device-Integration-Plugins

4.2. Architekturübersicht

4.3. Konzeption der wiederverwendbaren Routing-Infrastruktur

4.4. Registrierung und Lebenszyklusverwaltung

4.4.1. Integration Registry

4.4.2. Lease Monitor

4.4.3. Timeout, Reconnect und Deregistrierung

4.5. Multi-Service-Routing

4.5.1. Serviceidentifikation und Routenauflösung

4.5.2. Job-Korrelation

4.5.3. Fehler- und Timeoutbehandlung

4.6. Contracts und Job-Envelopes

4.7. Servicevertrag und SmartProxy

4.7.1. IDeviceIntegrationService

4.7.2. DeviceIntegrationServiceProxy

4.7.3. Code-first gRPC

4.8. TreeNode-Adapter

4.9. Subscription-Konzept

Kapitel 5.

Implementierung

5.1. Technologische Rahmenbedingungen

5.2. Projektstruktur

5.3. Implementierung der Routing-Grundlage

5.4. Implementierung der Job-, Ergebnis- und
Stream-Contracts

5.5. Implementierung des Multi-Service-Routings

5.6. Implementierung der Registrierung und
Lebenszyklusverwaltung

5.7. Implementierung des gRPC-Servicevertrags und
SmartProxys

5.8. Implementierung des Integration Clients

5.9. Implementierung des TreeNode-Adapters

5.10. Implementierung von Subscriptions

5.11. Einbindung in zenLAB

5.12. Anbindung eines Referenz-DeviceAgents

5.13. Herausforderungen während der Umsetzung

Kapitel 6.

Evaluation

6.1. Ziel der Evaluation

6.2. Versuchsaufbau und Evaluationskriterien

6.3. Überprüfung der funktionalen Anforderungen

6.3.1. Registrierung und eindeutige Adressierung

6.3.2. Routing mit mehreren Integrationen und Clients

6.3.3. Heartbeat, Timeout und Reconnect

6.3.4. TreeNode-Aufrufe und Subscriptions

6.3.5. Kompatibilität der bestehenden Client- und WebAPI-Grenze

6.4. Bewertung der Wiederverwendbarkeit und Erweiterbarkeit

6.5. Messung des Routing-Overheads

6.6. Bewertung anhand des Referenz-DeviceAgents

6.7. Diskussion der Ergebnisse

6.8. Grenzen der Evaluation

Kapitel 7.

Fazit und Ausblick

7.1. Zusammenfassung der Ergebnisse

7.2. Beantwortung der Forschungsfragen

7.3. Bewertung der Zielerreichung

7.4. Limitationen der Arbeit

7.5. Ausblick auf mögliche Erweiterungen

Anhang A.

Supplemental Information

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Und nun folgt – ob man es glaubt oder nicht – der dritte Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Nach diesem vierten Absatz beginnen wir eine neue Zählung. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Abbildungsverzeichnis

Tabellenverzeichnis

List of Listings

Literaturverzeichnis

- [1] K. Janning, F. Kaußmann, J. Krieger, J. Mathews, F. Nienhaus, D. Reibert, C. Stehmar, and C. Skazik-Voogt, “Labor 4.0 – next generation production in life sciences,” Fraunhofer-Institut für Produktionstechnologie IPT, Trendreport, 2025. [Online]. Available: <https://www.ipt.fraunhofer.de/content/dam/ipt/de/documents/whitepaper/fraunhofer-ipt-publikation-trendreport-labor-40-next-generation-production-in-life-science1.pdf>
- [2] L. Bromig, D. Leiter, A.-V. Mardale, N. von den Eichen, E. Bieringer, and D. Weuster-Botz, “The SiLA 2 manager for rapid device integration and workflow automation,” *SoftwareX*, vol. 17, p. 100991, 2022.
- [3] infoteam Software AG, “Unternehmen,” [Online]. Verfügbar: <https://infoteam.de/unternehmen>, zugriff am 31. Juli 2026.
- [4] —, “Life Science,” [Online]. Verfügbar: <https://infoteam.de/unsere-maerkte/life-science>, zugriff am 31. Juli 2026.
- [5] Microsoft, “Use hubs in asp.net core signalr,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/aspnet/core/signalr/hubs>, zugriff am 24. Juli 2026.
- [6] C. G. Gray and D. R. Cheriton, “Leases: An efficient fault-tolerant mechanism for distributed file cache consistency,” in *Proceedings of the 12th ACM Symposium on Operating Systems Principles*. ACM, 1989, pp. 202–210.
- [7] Microsoft, “Generate and consume async streams,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/generate-consume-asynchronous-stream>, zugriff am 24. Juli 2026.
- [8] —, “Code-first grpc services and clients with .net,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/aspnet/core/grpc/code-first>, zugriff am 24. Juli 2026.
- [9] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture: A System of Patterns*. Chichester, UK: Wiley, 1996, vol. 1.

- [10] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [11] Microsoft, “Use streaming in asp.net core signalr,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/aspnet/core/signalr/streaming>, zugriff am 24. Juli 2026.
- [12] —, “Dependency injection in .net,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection/overview>, zugriff am 24. Juli 2026.

Glossar

library A suite of reusable code inside of a programming language for software development. i

shell Terminal of a Linux/Unix system for entering commands. i